

In C programming language, the statement `X== -1;` generates the tokens:

Select one:

- a. `X -= - 1 ;`
- b. `X == - 1 ;`
- c. None of the mentioned answers.
- d. `X - = = - 1 ;`

e. `X == - 1 ;` ANSWER

For the operations `(.)` concatenation and the OR `(|)` operations, one of the following statements is TRUE. Select one:

- a. Both are commutative.
- b. Neither `(.)` nor `(|)` is commutative.
- c. `(.)` is commutative but `(|)` is not.
- d. **`(|)` is commutative but `(.)` is not. ANSWER**

The grammar is:

`N --> dN`

`N --> d`

- a. Not ambiguous
- b. none of the mentioned answers
- c. **Both not ambiguous and recursive. ANSWER**
- d. recursive.

Consider the if . . else grammar, G:

$S' \rightarrow S$

$S \rightarrow iSE \mid S \rightarrow a$

$E \rightarrow eS \mid E \rightarrow \lambda$

then

a. G is ambiguous because $\text{FIRST}(eS) \cap \text{FOLLOW}(E) = \phi$

b. G is ambiguous because E has two production

c. G is ambiguous because $\text{FIRST}(iSE) \cap \text{FIRST}(a) = \phi$

d. None of the mentioned answers

e. G is ambiguous because S has two production. ANSWER

Given the grammar: $S' \rightarrow S\$$ $S \rightarrow ABb \mid bc$ $A \rightarrow abAB \mid \lambda$ $B \rightarrow bc \mid cBS$, then:

a. $\text{FOLLOW}(S) = \{b, c, \$\}$. ANSWER

b. $\text{FOLLOW}(S) = \{\$ \}$

c. Both $\text{FOLLOW}(A) = \text{FOLLOW}(B)$ and $\text{FOLLOW}(S) = \{b, c, \$\}$

d. $\text{FOLLOW}(A) = \text{FOLLOW}(B)$

e. Both $\text{FOLLOW}(A) = \text{FOLLOW}(B)$ and $\text{FOLLOW}(S) = \{\$ \}$

.Given the grammar: $S \rightarrow R\$$ $R \rightarrow N.N$ $N \rightarrow dN \mid d$

Starting with the sentence $dd.d\$$, then in bottom up parsing technique the HANDLE is the

underlined in one of the following:

a. dd.d\$. ANSWER

b. dd.d\$

c. dd.d\$

d. None of the mentioned answers

The strings generated by the grammar: $S \rightarrow 1S1 \mid 0S0 \mid \lambda$

- a. None of the mentioned answers
- b. Equal number of 0's and 1's
- c. Strings of 1's followed by equal number of 0's
- d. Strings that reads the same from left and right. ANSWER**
- e. Strings of 0's followed by equal number of 1's

Given the alphabet $V = \{0, 1\}$, $L = \{00, 11\}$, then L^0 :

- a. $\{\Phi\}$
- b. λ
- c. $\{00, 11\}$
- d. $\{\lambda\}$. ANSWER**



In C programming language, "main" is considered as:

- a. Reserved(key) word.
- b. Standard identifier(name).**
- c. User defined name.

d. None of the mentioned answers. ANSWER

- e. Special token.

The productions in BNF:

$L \rightarrow L, N \mid N$

Using EBNF this is written as:

- a. $L \rightarrow (N, N)^*$
- b. None of the mentioned answers
- c. $L \rightarrow (, N)^* N$
- d. $L \rightarrow N(N)^*$ (ANSWER)**

The "stack overflow" error is:

- a. Semantic error
- b. Syntax error.
- c. runtime error. ANSWER**
- d. Both runtime and syntax error
- e. Both runtime and semantic error.

The syntax of the "if" statement in a PL means:

- a. The grammar of the "if" statement.
- b. The structure of the "if" statement.
- c. None of the mentioned answers.
- d. Both the grammar & structure of the "if" statement. ANSWER**

In LALR parsers, only one of the following is true:

a. A shift-reduce conflict may occur. ANSWER

- b. A shift-reduce or reduce-reduce conflict may occur
- c. None of the mentioned answers
- d. A reduce-reduce conflict may occur

e. A shift-shift conflict may occur

In regular expressions, the identity elements for the concatenation(.) and the OR(|) operations are:

a. λ & $\{\}$

b. $\{\lambda\}$ & $\{\phi\}$

c. None of the mentioned answers.

d. ϕ & $\{\lambda\}$ (ANSWER)

e. ϕ & λ

Given the grammar $A \rightarrow abX$ $A \rightarrow a$, using left factoring this grammar can be written as:

a. $A \rightarrow bX$ $A \rightarrow a$ $A \rightarrow \lambda$

b. $A \rightarrow aB$ $B \rightarrow bX$ $B \rightarrow \lambda$ (ANSWER)

c. $A \rightarrow aB$ $B \rightarrow bX$ $X \rightarrow \lambda$

d. $A \rightarrow abXB$ $X \rightarrow \lambda$ $B \rightarrow \lambda$

e. None of the mentioned answers

The regular expression which generates any number of 0's is followed by any number of 1's followed by any number of 2's is: Select one:

a. $(012)^*$

b. $(0|1|2)^*$

c. $(0|1)^*2^*$

d. $(0^*1^*2^*)$ (ANSWER)

Which of the following statements is TRUE: Select one:

a. The "array" type and the "assignment" statement are examples on "data" & "control" abstraction respectively. ANSWER

b. The "array" type and the "assignment" statement are examples on "control" & "data" abstraction respectively.

c. The "array" type and the "assignment" statement are examples on "control" abstraction only.

d. The "array" type and the "assignment" statement are examples on "data" abstraction only.

The code for the function "absolute value", ABSV in LISP: Select one:

a. `(defun ABSV [x] (if (< x 0) (- x) x))`

b. `(defun ABSV (x) (if (< x 0) then (- x) x))`

c. None of the mentioned answers. ANSWER

d. `(defun ABSV x (if (< x 0) (- x) x))`

e. `(defun ABSV (x) (if (< x 0) x (- x)))`

Handwritten notes in red:
defun. ABSV
(x (if...))

One of the following is not an example of static binding: Select one:

a. `float A[100];`

b. `const float bi=3.14;`

c. `scanf("%f", &X);` ANSWER

d. `float *X;`

e. None of the mentioned answers

If a PL has more instructions, then, the PL is:

- a. More complex.
- b. More orthogonal
- c. The number of instructions is irrelevant.
- d. None of the mentioned answers.
- e. Less orthogonal. ANSWER**

In LR parsers, the items: $[A \rightarrow b.aX, a]$ and $[B \rightarrow cA., b]$ cause: Select one:

- a. A reduce-reduce conflict on token b
- b. Do not cause any conflict
- c. A shift-reduce conflict on token a
- d. A shift-reduce conflict on token b. ANSWER**
- e. None of the mentioned answers

Given the following function in LISP:

`> (defun F(n m) (if(= m 1) n (+n (F n (-m 1))))) ,`

then the function call `>(F 4 6)` will print:

- a. -2
- b. 24 ANSWER**
- c. 10
- d. 2

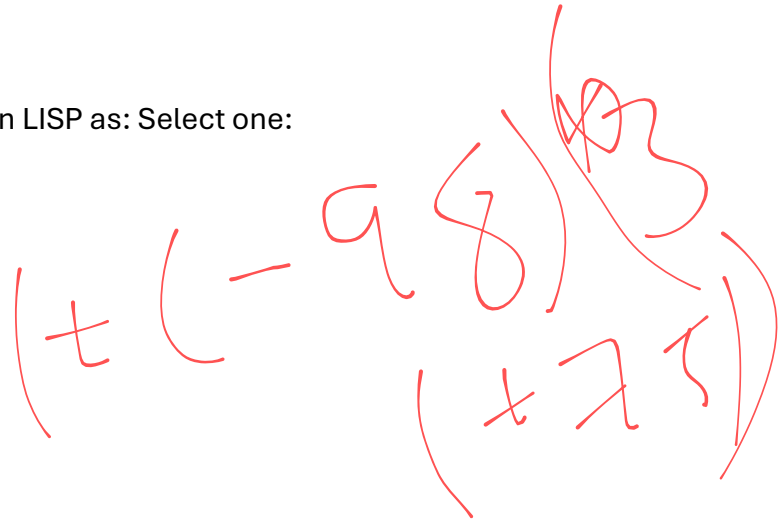
$4 \times 6 = 24$

The context free grammar $S \rightarrow 0S1 \mid 1S0 \mid \lambda$ generates strings with: Select one:

- a. Any number of 0's followed by same number of 1's
- b. Equal number of 0's and 1's. ANSWER**
- c. Unequal number of 0's and 1's
- d. Any number of 1's followed by same number of 0's

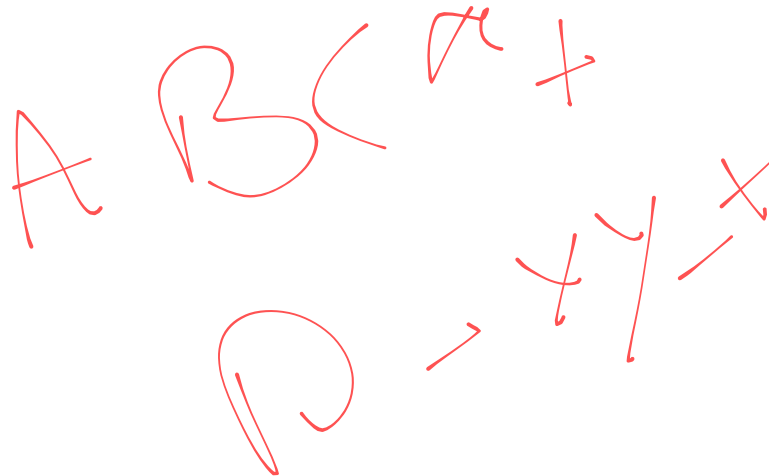
The expression $9-8+3*(7+5)$ is implemented in LISP as: Select one:

- a. None of the mentioned answers.
- b. $(+ (* (+ 7 5) 3) (- 9 8))$
- c. $(+ (- 9 8) (* 3 (+ 7 5)))$ ANSWER**
- d. $(* (+ (- 9 8) 3) (+ 7 5))$
- e. $(+ (- 9 8) (* (+ 7 5) 3))$

Handwritten red LISP expression: $(+ (- 9 8) (* 3 (+ 7 5)))$

The postfix expression corresponds to the infix expression: $A + B * C - D + (X - Y)$: Select one:

- a. $A B C * + X Y - D +$
- b. $A B C D X Y + * - + -$
- c. $B C * A + D - X Y - +$
- d. $A B C * + D - X Y - +$ ANSWER**
- e. None of the mentioned answers.

Handwritten red postfix expression: $A B C * + D - X Y - +$

Given the grammar: $E \rightarrow F-E \mid F \mid n$, then the value of $8 - 5 - 1$

equals: Select one:

- a. 2
- b. Can't exactly tell
- c. Error, the grammar is ambiguous
- d. None of the mentioned answers

e. 4 ANSWER

Left + Right
8 - 15 = -7

Given the alphabet $V=\{0,1\}$, then, $\{1\}^*$: Select one:

a. $\{1\}$

b. $\{ \}$ ANSWER

c. $\{\Phi\}$

d. $\{\lambda\}$

$\{1\}^* = \{1, 11, 111, \dots\}$
 $8 - 4 = 4$

Given the following left recursive grammar: $A \rightarrow Abc \mid a \mid b$, Eliminating

left recursion gives: Select one:

- a. $A \rightarrow aA' \mid A' \rightarrow bcA' \mid \lambda$
- b. $A \rightarrow cA' \mid A' \rightarrow bcA' \mid \lambda$
- c. None of the mentioned answers

d. $A \rightarrow aA' \mid A' \rightarrow bcA' \mid \lambda$ ANSWER

Given the grammar

$S' \rightarrow S \$$

$S \rightarrow (L) \mid a$

$L \rightarrow SA$

$A \rightarrow (, S)^* A; \mid \lambda$

Note: The parenthesis (,) in the production $A \rightarrow (, S)^* A;$ are not grammar symbols. Then, FOLLOW(S) = Select one:

a. None of the mentioned answers (ANSWER)

b. { \$, ; }

c. { \$, ; }

d. { \$ }

e. { \$, }

The reliability issue of a PL is affected by the factors: Select one:

a. Exception handling.

b. None of the mentioned answers.

c. Type checking.

d. Both type checking & exception handling. ANSWER

The imperative PL using the following runtime environment. Select one:

a. None of the mentioned answers.

b. Fully static.

c. Stack-Based. ANSWER

d. Fully dynamic.

Which of the following is a dynamic binding:

a. `Int A[10];`

b. `Float x;`

c. All of the mentioned answers

d. None of the mentioned answers

e. `Int *p;` **ANSWER**

↳ Static binding
new lp is dynamic
amir